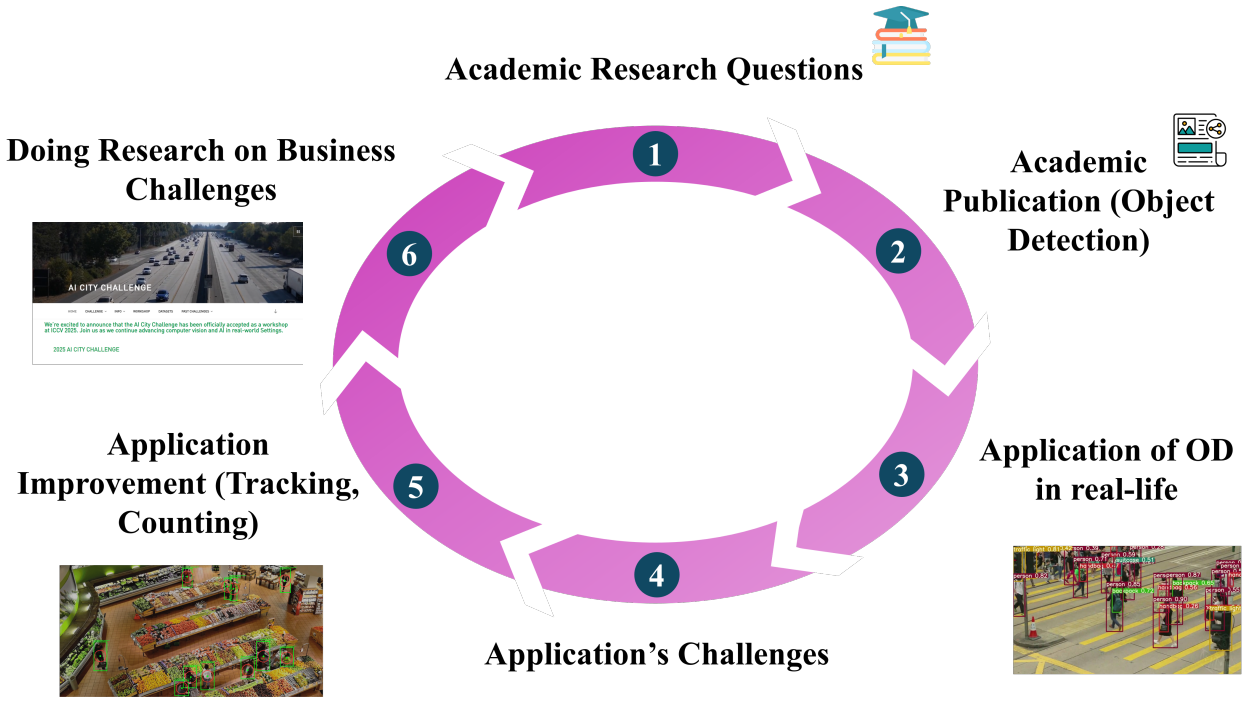
AI VIET NAM – AIO COURSE 2025

Tutorial: Giới thiệu về Object Tracking và Counting

Dang-Nha Nguyen Dinh-Vinh Nguyen Quang-Vinh Nguyen

I Mở đầu

Trong hệ sinh thái AI hiện nay, sự giao thoa giữa nghiên cứu hàn lâm và ứng dụng thực tiễn đang được thúc đẩy và diễn ra cực kỳ nhanh chóng. Đặc biệt trong lĩnh vực như **Computer Vision**, các mô hình lý thuyết thường nhanh chóng được chuyển hóa thành các giải pháp thực tế ngay sau khi công bố. Song, trong khi các nghiên cứu tập trung vào việc phát triển các mô hình với độ chính xác và hiệu năng cao hơn, các ứng dụng lại khai thác những mô hình này để giải quyết các bài toán thực tiễn trong nhiều lĩnh vực khác nhau.



Hình 1: Sự gắn kết giữa nghiên cứu và thực tiễn đặt nền móng sự phát triển vượt trội của trí tuệ nhân tạo.

Một trong những bài toán nền tảng của Computer Vision là **Object Detection** – nhiệm vụ xác định và định vị các đối tượng xuất hiện trong hình ảnh. Nhờ sự phát triển của các mô hình học sâu, đặc biệt là các kiến trúc như YOLO, Faster R-CNN hay RetinaNet, object detection đã đạt được những bước tiến vượt bậc và được ứng dụng rộng rãi trong nhiều hệ thống thực tế. Tuy nhiên, trong nhiều bài toán thực tế, việc chỉ biết đối tượng nào xuất hiện trong một bức ảnh là chưa đủ. Thay vào đó, chúng ta cần

hiểu đối tượng đó di chuyển và thay đổi như thế nào theo thời gian. Nói cách khác, thay vì chỉ phân tích từng hình ảnh riêng lẻ, hệ thống cần có khả năng phân tích chuỗi các frame trong video để hiểu được hành vi và quỹ đạo của các đối tượng.

Ví dụ, trong các hệ thống giám sát giao thông, camera được sử dụng để theo dõi lưu lượng phương tiện và thống kê số lượng xe đi qua một giao lộ. Trong lĩnh vực bán lẻ, camera có thể được dùng để phân tích hành vi khách hàng, chẳng hạn như thời gian khách hàng di chuyển trong cửa hàng hoặc thời gian dừng lại trước một khu vực trưng bày sản phẩm. Những bài toán này đều có thể bắt đầu từ object detection, nhưng để khai thác được thông tin hữu ích hơn, hệ thống cần khả năng theo dõi các đối tượng xuyên suốt nhiều frame.

Các mô hình object detection hiện đại có thể xử lý từng frame của video và dự đoán bounding boxes cho các đối tượng như người, xe cộ hoặc động vật. Tuy nhiên, detector chỉ trả lời câu hỏi: *"Những đối tượng nào xuất hiện trong frame này?"* Nó không thể trả lời các câu hỏi quan trọng hơn như: Liệu hai bounding boxes ở hai frame liên tiếp có thuộc cùng một đối tượng hay không? Có bao nhiêu đối tượng duy nhất đã xuất hiện trong toàn bộ chuỗi video? ... Nếu chúng ta áp dụng object detection một cách độc lập cho từng frame, hệ thống có thể phát hiện cùng một đối tượng nhiều lần nhưng không thể xác định liệu các phát hiện đó có tương ứng với cùng một thực thể trong thế giới thực hay không. Đây chính là hạn chế quan trọng của việc chỉ sử dụng detection khi xử lý dữ liệu video.

Để giải quyết vấn đề này, **Object Tracking** được giới thiệu nhằm gán một danh tính nhất quán (identity) cho mỗi đối tượng qua các frame liên tiếp. Nhờ đó, hệ thống có thể theo dõi chuyển động của đối tượng theo thời gian, tái tạo quỹ đạo di chuyển và phân tích các hành vi liên quan. Khi các đối tượng đã được theo dõi ổn định, nhiều bài toán ở mức cao hơn trở nên khả thi. Một ví dụ phổ biến là **Object Counting**, trong đó hệ thống đếm số lượng đối tượng khi chúng đi qua một đường ảo (virtual line) hoặc đi vào một vùng quan tâm (region of interest). Phương pháp này được ứng dụng rộng rãi trong các hệ thống đếm xe, đếm người, giám sát sản xuất và nhiều lĩnh vực khác.

Trong bài viết này, chúng ta sẽ làm rõ mối quan hệ giữa object detection, object tracking và object counting, đồng thời tìm hiểu các thành phần kỹ thuật đứng sau các hệ thống này. Bài viết sẽ bắt đầu từ các phương pháp **classical single object tracking**, sau đó mở rộng sang các thuật toán **modern multiple object tracking** như **Deep SORT** và **ByteTrack**. Cuối cùng, chúng ta sẽ kết hợp các thành phần này để xây dựng một pipeline hoàn chỉnh cho bài toán đếm phương tiện trong video giao thông.

Mục lục

I Mở đầu **1**
1 Thảo luận từ bài toán Object Detection sang Object Tracking 4
2 Các kiểu Object Tracking 6

II Single Object Tracking (Tracking by Method) **8**
1 Template Matching Tracking 8
2 Histogram Matching Tracking 10
3 Mean Shift Tracking 11

III Multiple Object Tracking (Tracking by Detection) **14**
1 Tổng quan 14
2 Motion Model 15
3 Data Association 17

IV Modern MOT Algorithms **18**
1 ByteTrack 18
2 DeepSORT 19

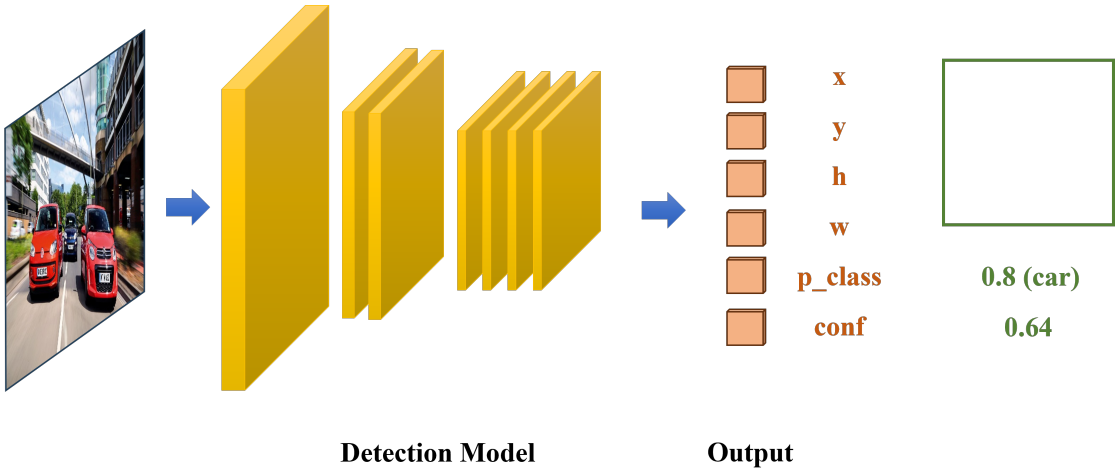
V Giới thiệu về bài tập Object Tracking & Counting **20**
1 Giới thiệu các thư viện 21
2 Ví dụ triển khai 22

VI Phụ Lục **23**

1 Thảo luận từ bài toán Object Detection sang Object Tracking

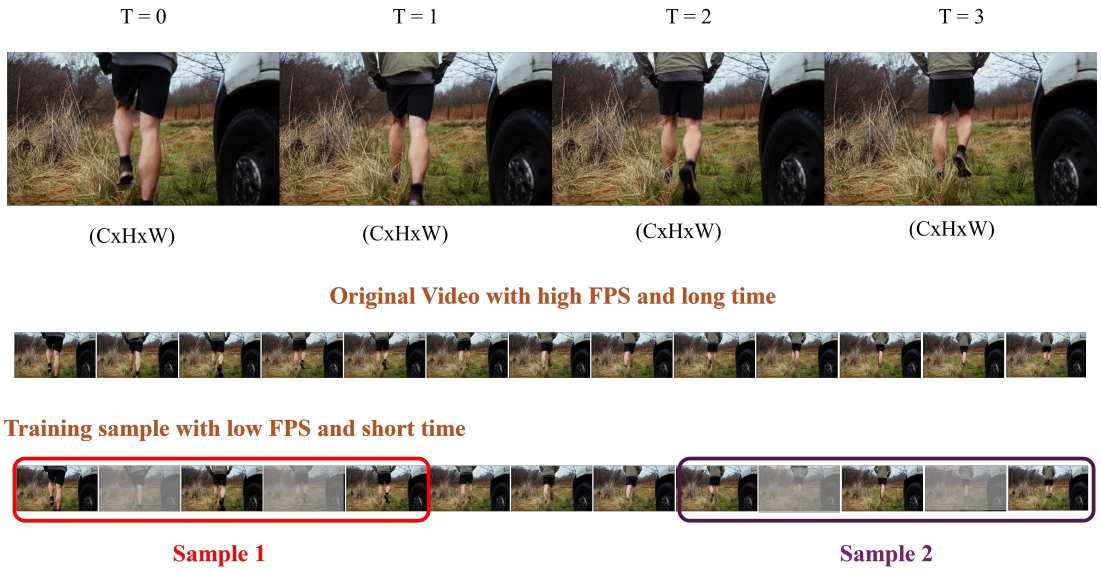
1.1 Object Detection đối với ảnh, video

Object Detection là một trong những bài toán cơ bản, phổ biến của Computer Vision, với mục tiêu xác định sự xuất hiện và vị trí của các đối tượng trong một hình ảnh. Cho một ảnh đầu vào, mô hình object detection sẽ dự đoán một tập các bounding boxes tương ứng với vị trí của các đối tượng, đồng thời gán nhãn lớp (class label) và một giá trị độ tin cậy (confidence score) cho mỗi dự đoán. Nhờ sự phát triển của các mô hình học sâu, đặc biệt là các kiến trúc như YOLO, Faster R-CNN hay RetinaNet, object detection hiện nay có thể đạt độ chính xác cao và tốc độ xử lý gần thời gian thực.



Hình 2: Ví dụ về Object Detection

Khi áp dụng cho video, object detection thường được triển khai bằng cách xử lý từng frame của video một cách độc lập. Một video có thể được xem như một chuỗi các hình ảnh liên tiếp theo thời gian, vì vậy cùng một mô hình detection có thể được áp dụng cho từng frame để phát hiện các đối tượng xuất hiện trong đó. Kết quả thu được là một tập các bounding boxes cho mỗi frame, biểu diễn vị trí của các đối tượng như người, phương tiện hoặc động vật tại thời điểm tương ứng.



Hình 3: Ví dụ về dữ liệu video định dạng từng frame ảnh.

Tuy nhiên, cách tiếp cận này chỉ cung cấp thông tin về sự xuất hiện của các đối tượng trong từng frame riêng lẻ. Mặc dù cùng một đối tượng có thể xuất hiện trong nhiều frame liên tiếp, các dự đoán từ detector không chứa thông tin nào cho biết liệu hai bounding boxes ở hai frame khác nhau có thuộc về cùng một đối tượng trong thế giới thực hay không. Do đó, khi chỉ sử dụng object detection, hệ thống chỉ có thể trả lời câu hỏi "đối tượng nào xuất hiện trong frame này", nhưng không thể suy luận được sự liên tục của đối tượng theo thời gian.

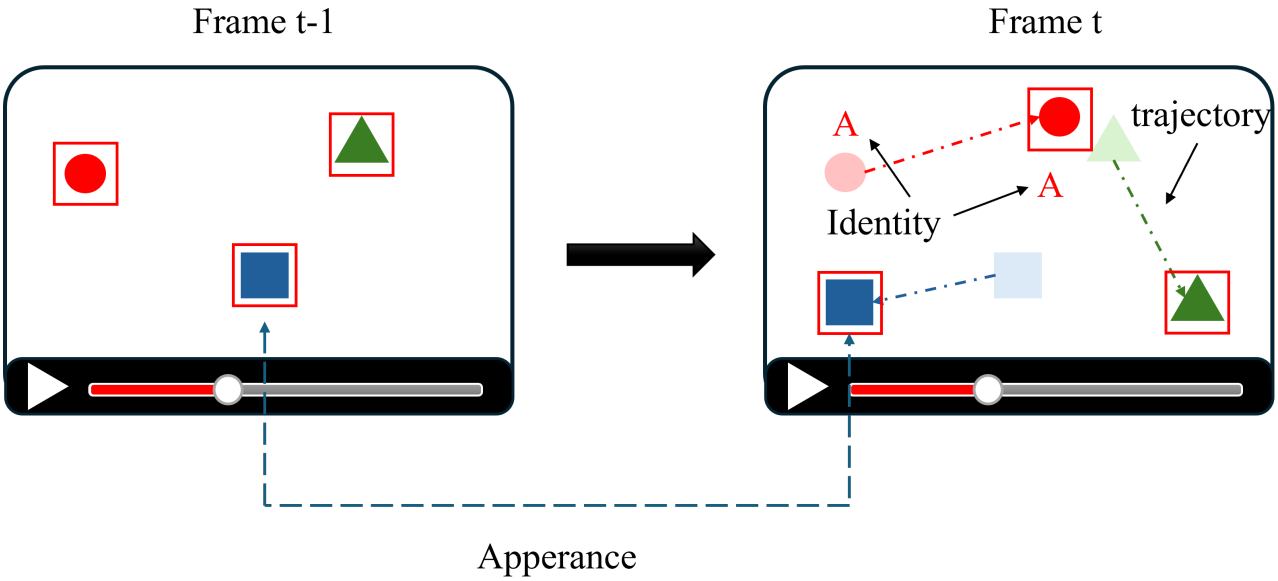
1.2 Association Problem

Hạn chế trên dẫn đến một vấn đề quan trọng khi xử lý dữ liệu video, thường được gọi là bài toán association. Khi một detector được áp dụng cho từng frame, hệ thống sẽ thu được nhiều bounding boxes cho mỗi thời điểm. Tuy nhiên, để hiểu được chuyển động của các đối tượng theo thời gian, chúng ta cần xác định mối liên hệ giữa các bounding boxes ở các frame khác nhau. Nói cách khác, hệ thống cần biết bounding box nào ở frame hiện tại tương ứng với đối tượng nào đã xuất hiện ở frame trước đó. Bài toán association trở nên phức tạp hơn trong các kịch bản thực tế. Các đối tượng có thể di chuyển nhanh, thay đổi vị trí, che khuất lẫn nhau (occlusion) hoặc thậm chí tạm thời biến mất khỏi khung hình. Ngoài ra, các detector cũng có thể tạo ra false positives hoặc bỏ sót một số đối tượng trong một số frame nhất định. Những yếu tố này khiến việc xác định sự tương ứng giữa các bounding boxes qua các frame trở thành một bài toán không đơn giản.
Về bản chất, bài toán association có thể được xem như một quá trình so khớp (matching) giữa các detections ở frame hiện tại và các đối tượng đã được quan sát trước đó. Quá trình này thường dựa trên nhiều nguồn thông tin khác nhau, chẳng hạn như vị trí không gian của bounding boxes, đặc trưng hình ảnh của đối tượng, hoặc mô hình chuyển động (motion model) theo thời gian. Việc giải quyết chính xác bài toán association là yếu tố then chốt để có thể theo dõi các đối tượng một cách ổn định trong video.

1.3 Object Tracking

Để giải quyết bài toán association và duy trì sự liên tục của các đối tượng qua thời gian, khái niệm Object Tracking được đưa ra. Trong Computer Vision, object tracking là nhiệm vụ gán một danh tính nhất quán (identity) cho mỗi đối tượng khi nó xuất hiện qua nhiều frame liên tiếp trong một video.

Thay vì chỉ phát hiện các đối tượng trong từng frame riêng lẻ, hệ thống tracking còn phải duy trì một track cho mỗi đối tượng, biểu diễn lịch sử vị trí của nó theo thời gian.



Hình 4: Ví dụ về association trong Object Tracking

Kết quả của một hệ thống tracking thường bao gồm không chỉ bounding box của đối tượng, mà còn một track ID duy nhất được gán cho đối tượng đó. Nhờ track ID này, hệ thống có thể nhận biết rằng một đối tượng xuất hiện ở các frame khác nhau thực chất là cùng một thực thể. Theo thời gian, chuỗi các vị trí của cùng một track ID sẽ tạo thành quỹ đạo di chuyển (trajectory) của đối tượng trong video. Nhờ khả năng duy trì danh tính của các đối tượng, object tracking cho phép hệ thống khai thác nhiều thông tin ở mức cao hơn so với detection đơn thuần. Ví dụ, từ các trajectories của đối tượng, ta có thể phân tích hành vi chuyển động, tương tác giữa các đối tượng, hoặc các sự kiện xảy ra trong video. Một ứng dụng phổ biến là object counting, trong đó hệ thống đếm số lượng đối tượng khi quỹ đạo của chúng cắt qua một đường ảo hoặc đi vào một vùng quan tâm trong khung hình.
Tổng kết lại, nếu object detection trả lời câu hỏi "đối tượng nào xuất hiện trong frame này", thì object tracking mở rộng bài toán để trả lời "đối tượng đó di chuyển như thế nào theo thời gian". Chính sự chuyển đổi từ phân tích từng hình ảnh riêng lẻ sang phân tích chuỗi hình ảnh theo thời gian đã mở ra khả năng xây dựng các hệ thống phân tích video thông minh ứng dụng trong nhiều lĩnh vực khác nhau.

2 Các kiểu Object Tracking

2.1 Single Object Tracking

Single Object Tracking (SOT) là bài toán theo dõi một đối tượng duy nhất xuyên suốt một chuỗi frame trong video. Trong thiết lập, vị trí ban đầu của đối tượng thường được cung cấp sẵn ở frame đầu tiên, thường dưới dạng một bounding box xác định vùng chứa đối tượng cần theo dõi. Nhiệm vụ của hệ thống tracking là dự đoán vị trí của cùng đối tượng đó trong các frame tiếp theo, bất chấp sự thay đổi về vị trí, kích thước hoặc góc nhìn. Đây chính là bài toán khởi đầu cho các bài toán tracking phức tạp sau này.
Khác với object detection, nơi mô hình phải phát hiện tất cả các đối tượng trong một hình ảnh, bài toán single object tracking chỉ tập trung vào một mục tiêu cụ thể đã được xác định trước. Điều này giúp giảm đáng kể độ phức tạp của bài toán, bởi hệ thống không cần phải phân biệt giữa nhiều đối

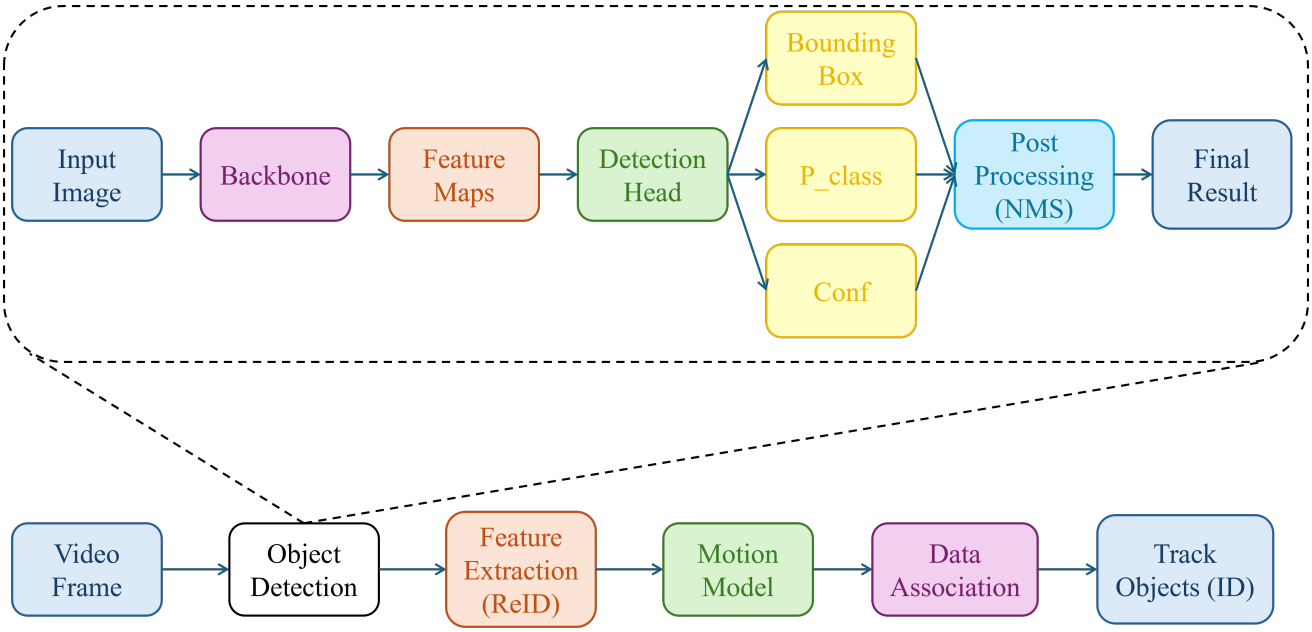
tượng khác nhau hay duy trì danh tính cho nhiều mục tiêu cùng lúc. Thay vào đó, trọng tâm của SOT nằm ở việc duy trì sự tương đồng về đặc trưng (appearance consistency) của đối tượng theo thời gian. Trong lịch sử phát triển của Computer Vision, nhiều phương pháp cổ điển đã được đề xuất để giải quyết bài toán single object tracking. Các phương pháp này thường dựa trên việc so khớp đặc trưng giữa các frame liên tiếp, ví dụ như template matching, histogram matching, hoặc các kỹ thuật tối ưu hóa như Mean Shift tracking. Mặc dù các phương pháp này tương đối đơn giản so với các mô hình học sâu hiện đại, nhưng như đã đề cập ở trước, chúng đóng vai trò cực kỳ quan trọng trong việc hình thành nền tảng cho các hệ thống tracking sau này.

2.2 Multiple Object Tracking

Trong khi single object tracking chỉ tập trung vào một mục tiêu duy nhất, Multiple Object Tracking (MOT) mở rộng bài toán để theo dõi nhiều đối tượng đồng thời trong cùng một video. Trong thiết lập này, hệ thống không chỉ cần xác định vị trí của các đối tượng trong từng frame mà còn phải duy trì danh tính nhất quán cho từng đối tượng qua thời gian. Điều này có nghĩa là mỗi đối tượng phải được gán một track ID riêng biệt, cho phép hệ thống nhận biết và theo dõi quỹ đạo của từng đối tượng trong suốt chuỗi video.

So với single object tracking, MOT là một bài toán phức tạp hơn đáng kể. Trước hết, hệ thống phải xử lý nhiều đối tượng xuất hiện cùng lúc, có thể di chuyển theo các hướng khác nhau và tương tác với nhau trong không gian. Ngoài ra, các đối tượng có thể che khuất lẫn nhau, đi vào hoặc rời khỏi khung hình, hoặc thậm chí có ngoại hình tương tự nhau, khiến việc duy trì danh tính trở nên khó khăn. Những yếu tố này làm cho bài toán MOT không chỉ là vấn đề dự đoán vị trí, mà còn là một bài toán quản lý và duy trì danh tính đối tượng theo thời gian.

Trong các hệ thống hiện đại, bài toán multiple object tracking thường được giải quyết theo một mô hình phổ biến gọi là tracking-by-detection. Trong cách tiếp cận này, một mô hình object detection được sử dụng để phát hiện các đối tượng trong từng frame của video. Sau đó, một thuật toán tracking sẽ thực hiện bước data association, tức là xác định sự tương ứng giữa các detections ở frame hiện tại và các đối tượng đã được theo dõi trước đó. Kết quả của quá trình này là một tập các track, mỗi track biểu diễn quỹ đạo chuyển động của một đối tượng trong video.



Hình 5: So sánh pipeline của object detection và object tracking

Nhiều thuật toán hiện đại đã được phát triển dựa trên mô hình tracking-by-detection nhằm cải thiện độ chính xác và tính ổn định của hệ thống tracking. Các phương pháp như Deep SORT sử dụng thêm đặc trưng hình ảnh để tăng độ tin cậy trong quá trình association, trong khi các thuật toán gần đây như ByteTrack tập trung tối ưu hóa quá trình matching giữa các detections nhằm giảm thiểu lỗi mất track hoặc thay đổi ID. Những phương pháp này đã trở thành nền tảng cho nhiều hệ thống phân tích video hiện đại trong các lĩnh vực như giám sát giao thông, phân tích hành vi đám đông và hệ thống an ninh thông minh.

II Single Object Tracking (Tracking by Method)

1 Template Matching Tracking

1.1 Problem Formulation

Giả sử một video được biểu diễn dưới dạng một chuỗi các frame theo thời gian:

$$I_1, I_2, \ldots, I_T \tag{1}$$

trong đó I_t biểu diễn hình ảnh tại thời điểm t. Trong bài toán *Single Object Tracking*, vị trí ban đầu của đối tượng trong frame đầu tiên được cung cấp dưới dạng một bounding box:

$$B_1 = (x_1, y_1, w, h) \tag{2}$$

trong đó (x_1, y_1) là tọa độ góc trên bên trái của bounding box, còn w và h lần lượt là chiều rộng và chiều cao của vùng chứa đối tượng.
Mục tiêu của bài toán tracking là xác định vị trí của cùng đối tượng này trong các frame tiếp theo. Cụ thể, với mỗi frame I_t, ta cần ước lượng bounding box:

$$B_t = (x_t, y_t, w, h) \tag{3}$$

sao cho vùng ảnh tương ứng trong frame I_t giống nhất với đối tượng ban đầu.

1.2 Template Extraction

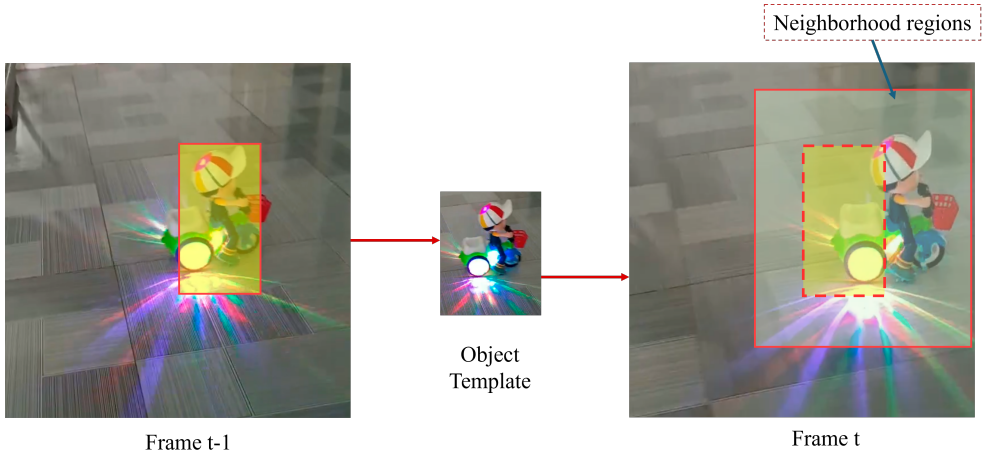
Trong phương pháp template matching, đối tượng được biểu diễn bởi một *template* được trích xuất từ frame đầu tiên. Template này là một vùng ảnh con chứa đối tượng cần theo dõi:

$$T = I_1[x_1 : x_1 + w,\ y_1 : y_1 + h] \tag{4}$$

Trong đó T là một ma trận kích thước $w \times h$ chứa các giá trị pixel của đối tượng. Template đóng vai trò như một biểu diễn ngoại hình (*appearance representation*) của đối tượng.

1.3 Search Region

Một cách tiếp cận trực tiếp là quét template trên toàn bộ frame để tìm vị trí phù hợp nhất. Tuy nhiên cách làm này có chi phí tính toán lớn và thường không cần thiết, bởi trong nhiều trường hợp đối tượng chỉ di chuyển một khoảng nhỏ giữa hai frame liên tiếp.
Do đó, nhiều hệ thống template matching định nghĩa một *search region* S_t xung quanh vị trí của đối tượng ở frame trước đó. Nếu vị trí của đối tượng ở frame $t-1$ là (x_{t-1}, y_{t-1}), vùng tìm kiếm được đặt xung quanh vị trí này nhằm giảm không gian tìm kiếm và tăng tốc quá trình tracking.



Hình 6: Ví dụ về search region - neighborhood region

1.4 Similarity Functions

Để xác định vị trí của đối tượng trong frame mới, thuật toán cần đo lường mức độ tương đồng giữa template T và các vùng ảnh ứng viên trong frame I_t. Một trong những phương pháp phổ biến là *Sum of Squared Differences (SSD)*:

$$SSD(x,y) = \sum_{i=0}^{w-1}\sum_{j=0}^{h-1}(T(i,j) - I_t(x+i, y+j))^2 \tag{5}$$

Giá trị SSD càng nhỏ thì mức độ tương đồng giữa hai vùng ảnh càng cao. Vị trí của đối tượng được xác định bằng cách:

$$(x_t, y_t) = \arg\min_{(x,y)\in S_t} SSD(x,y) \tag{6}$$

Một phương pháp khác thường được sử dụng là *Normalized Cross-Correlation (NCC)*:

$$NCC(x,y) = \frac{\sum_{i,j}(T(i,j) - \bar{T})(I_t(x+i, y+j) - \bar{I})}{\sqrt{\sum_{i,j}(T(i,j) - \bar{T})^2}\sqrt{\sum_{i,j}(I_t(x+i, y+j) - \bar{I})^2}} \tag{7}$$

Trong đó $\bar{T}$ và $\bar{I}$ lần lượt là giá trị trung bình của template và vùng ảnh ứng viên. Trong trường hợp này, vị trí của đối tượng được xác định bằng cách tối đa hóa giá trị NCC.

1.5 Target Localization

Sau khi tính toán similarity cho tất cả các vị trí ứng viên trong vùng tìm kiếm, thuật toán xây dựng một *similarity map*. Vị trí của đối tượng được xác định tại điểm có độ tương đồng cao nhất:

$$(x_t, y_t) = \arg\max_{(x,y)\in S_t} s(x,y) \tag{8}$$

trong đó $s(x,y)$ là giá trị similarity tại vị trí (x,y). Bounding box của đối tượng trong frame t được cập nhật thành:

$$B_t = (x_t, y_t, w, h) \tag{9}$$

1.6 Computational Complexity

Nếu template có kích thước $w \times h$ và frame có kích thước $W \times H$, việc quét template trên toàn bộ frame sẽ có độ phức tạp:

$$O(W \times H \times w \times h) \tag{10}$$

Chi phí tính toán này có thể trở nên lớn đối với video có độ phân giải cao. Vì vậy trong thực tế, các hệ thống tracking thường giới hạn vùng tìm kiếm hoặc sử dụng các kỹ thuật tối ưu hóa để tăng tốc quá trình so khớp.

1.7 Limitations

Mặc dù đơn giản và dễ triển khai, template matching tồn tại một số hạn chế quan trọng. Phương pháp này giả định rằng ngoại hình của đối tượng không thay đổi nhiều theo thời gian, trong khi trong thực tế đối tượng có thể thay đổi góc nhìn, kích thước hoặc điều kiện ánh sáng. Ngoài ra, template cố định ban đầu có thể không còn đại diện tốt cho đối tượng sau một khoảng thời gian dài, và phương pháp cũng nhạy cảm với hiện tượng che khuất (*occlusion*).

2 Histogram Matching Tracking

2.1 Problem Formulation

Tương tự như template matching, giả sử một video được biểu diễn dưới dạng chuỗi các frame theo thời gian:

$$I_1, I_2, \ldots, I_T \tag{11}$$

Trong bài toán *Single Object Tracking*, vị trí ban đầu của đối tượng trong frame đầu tiên được cung cấp dưới dạng một bounding box:

$$B_1 = (x_1, y_1, w, h) \tag{12}$$

Mục tiêu của thuật toán là xác định vị trí của đối tượng trong các frame tiếp theo bằng cách tìm vùng ảnh có đặc trưng ngoại hình giống nhất với đối tượng ban đầu. Khác với template matching sử dụng giá trị pixel trực tiếp, histogram matching sử dụng phân bố thống kê của các đặc trưng ảnh, thường là màu sắc.

2.2 Histogram Representation

Trong phương pháp này, đối tượng được biểu diễn bằng histogram của một đặc trưng ảnh, thường là màu sắc trong không gian RGB hoặc HSV. Giả sử histogram của đối tượng có K bins, ta định nghĩa histogram của template như sau:

$$H_T = (h_1, h_2, \ldots, h_K) \tag{13}$$

Trong đó h_k biểu diễn số lượng pixel của đối tượng thuộc bin thứ k. Để tránh phụ thuộc vào kích thước vùng ảnh, histogram thường được chuẩn hóa:

$$p_k = \frac{h_k}{\sum_{j=1}^{K} h_j} \tag{14}$$

Khi đó vector $P = (p_1, p_2, \ldots, p_K)$ được xem như một phân phối xác suất biểu diễn đặc trưng ngoại hình của đối tượng.

2.3 Candidate Histogram

Trong mỗi frame mới I_t, thuật toán xét các vùng ảnh ứng viên trong vùng tìm kiếm S_t. Với mỗi vị trí ứng viên (x, y), ta trích xuất một vùng ảnh kích thước $w \times h$ và tính histogram của vùng này:

$$Q(x, y) = (q_1, q_2, \ldots, q_K) \tag{15}$$

Histogram này cũng được chuẩn hóa để tạo thành một phân phối xác suất:

$$q_k = \frac{g_k}{\sum_{j=1}^{K} g_j} \tag{16}$$

trong đó g_k là số lượng pixel thuộc bin thứ k trong vùng ứng viên.

2.4 Similarity Measurement

Sau khi có histogram của template và histogram của vùng ứng viên, bước tiếp theo là đo lường mức độ tương đồng giữa hai phân phối này. Một thước đo phổ biến là Bhattacharyya coefficient, được định nghĩa như sau:

$$\rho(P, Q) = \sum_{k=1}^{K} \sqrt{p_k q_k} \tag{17}$$

Giá trị ρ nằm trong khoảng $[0, 1]$. Khi hai histogram giống nhau hoàn toàn, giá trị này đạt cực đại bằng 1. Do đó vị trí của đối tượng trong frame t được xác định bằng cách:

$$(x_t, y_t) = \arg \max_{(x,y) \in S_t} \rho(P, Q(x, y)) \tag{18}$$

Sau khi tìm được vị trí tối ưu, bounding box của đối tượng được cập nhật:

$$B_t = (x_t, y_t, w, h) \tag{19}$$

2.5 Thảo luận

So với template matching dựa trực tiếp vào pixel, histogram matching có một số ưu điểm đáng chú ý. Do histogram chỉ mô tả phân bố thống kê của đặc trưng ảnh, phương pháp này ít nhạy cảm hơn với các thay đổi nhỏ về vị trí pixel hoặc nhiễu ảnh. Ngoài ra, khi sử dụng histogram màu trong không gian HSV, thuật toán cũng có khả năng chống chịu tốt hơn trước sự thay đổi về ánh sáng.

Tuy nhiên, histogram matching cũng tồn tại một số hạn chế. Vì histogram chỉ biểu diễn phân bố màu mà không lưu thông tin về cấu trúc không gian của ảnh, hai vùng ảnh có phân bố màu tương tự nhưng hình dạng khác nhau vẫn có thể được xem là giống nhau. Điều này có thể dẫn đến sai lệch trong quá trình tracking khi trong khung hình tồn tại nhiều đối tượng có màu sắc tương tự.

Ngoài ra, phương pháp này cũng gặp khó khăn khi đối tượng thay đổi mạnh về ngoại hình, bị che khuất hoặc khi môi trường xung quanh có màu sắc gần giống với đối tượng cần theo dõi.

3 Mean Shift Tracking

3.1 Motivation

Các phương pháp như template matching hoặc histogram matching thường cần quét toàn bộ vùng tìm kiếm để xác định vị trí của đối tượng. Điều này có thể dẫn đến chi phí tính toán lớn, đặc biệt khi video có độ phân giải cao.

Mean Shift Tracking được đề xuất như một phương pháp hiệu quả hơn để tìm vị trí của đối tượng bằng cách xem bài toán tracking như một bài toán tối ưu hóa. Thay vì kiểm tra tất cả các vị trí khả dĩ, thuật toán sử dụng một phương pháp lặp để dịch chuyển cửa sổ tìm kiếm về phía vùng có độ tương đồng cao hơn giữa đặc trưng của đối tượng và vùng ảnh hiện tại.
Ý tưởng cốt lõi của mean shift là tìm cực đại của một hàm mật độ xác suất trong không gian đặc trưng. Trong bối cảnh tracking, cực đại này tương ứng với vị trí có sự tương đồng cao nhất giữa đặc trưng của đối tượng và vùng ảnh trong frame hiện tại.

3.2 Target Representation

Giống như histogram matching, mean shift tracking thường biểu diễn đối tượng bằng histogram của một đặc trưng ảnh, phổ biến nhất là màu sắc. Giả sử histogram của đối tượng có K bins. Histogram chuẩn hóa của đối tượng được ký hiệu là:

$$P = (p_1, p_2, \ldots, p_K) \tag{20}$$

trong đó:

$$\sum_{k=1}^{K} p_k = 1 \tag{21}$$

Histogram này được tính từ các pixel trong bounding box ban đầu. Tuy nhiên, để tăng tính ổn định, các pixel thường được gán trọng số theo khoảng cách tới tâm của bounding box bằng một hàm kernel. Điều này giúp các pixel gần tâm của đối tượng có ảnh hưởng lớn hơn đến histogram.

3.3 Kernel Density Estimation

Giả sử bounding box ban đầu chứa n pixel, mỗi pixel có vị trí x_i. Histogram của đối tượng có thể được ước lượng bằng kernel density estimation:

$$p_k = C \sum_{i=1}^{n} K\left(\left\|\frac{x_i}{h}\right\|^2\right) \delta[b(x_i) - k] \tag{22}$$

trong đó:

- $K(\cdot)$ là kernel function
- h là bandwidth của kernel
- $b(x_i)$ ánh xạ pixel x_i tới bin histogram
- δ là hàm Kronecker
- C là hằng số chuẩn hóa

Kernel phổ biến nhất trong tracking là Epanechnikov kernel, bởi vì nó tối ưu về mặt tính toán và thường được sử dụng trong các phương pháp mean shift.

3.4 Candidate Model

Trong mỗi frame mới, với một vị trí cửa sổ tìm kiếm y, ta tính histogram của vùng ảnh ứng viên:

$$Q(y) = (q_1(y), q_2(y), \ldots, q_K(y)) \tag{23}$$

Histogram này cũng được tính bằng kernel density estimation:

$$q_k(y) = C \sum_{i=1}^{n} K\left(\left\|\frac{x_i - y}{h}\right\|^2\right) \delta[b(x_i) - k] \tag{24}$$

Trong đó y là tâm của cửa sổ tìm kiếm.

3.5 Similarity Function

Để đo lường mức độ tương đồng giữa histogram của đối tượng và histogram của vùng ứng viên, mean shift tracking sử dụng Bhattacharyya coefficient:

$$\rho(P, Q(y)) = \sum_{k=1}^{K} \sqrt{p_k q_k(y)} \tag{25}$$

Giá trị này càng lớn thì hai histogram càng giống nhau. Do đó, bài toán tracking có thể được biểu diễn như sau:

$$y_t = \arg\max_y \rho(P, Q(y)) \tag{26}$$

3.6 Mean Shift Update

Để tìm cực đại của hàm tương đồng trên, mean shift sử dụng một quy tắc cập nhật lặp. Từ phép xấp xỉ Taylor của Bhattacharyya coefficient, ta thu được trọng số của mỗi pixel:

$$w_i = \sum_{k=1}^{K} \frac{p_k}{q_k(y)} \delta[b(x_i) - k] \tag{27}$$

Các trọng số này thể hiện mức độ đóng góp của từng pixel vào sự tương đồng giữa hai histogram. Vector mean shift được tính như sau:

$$y_{new} = \frac{\sum_{i=1}^{n} x_i w_i g\left(\left\|\frac{x_i - y}{h}\right\|^2\right)}{\sum_{i=1}^{n} w_i g\left(\left\|\frac{x_i - y}{h}\right\|^2\right)} \tag{28}$$

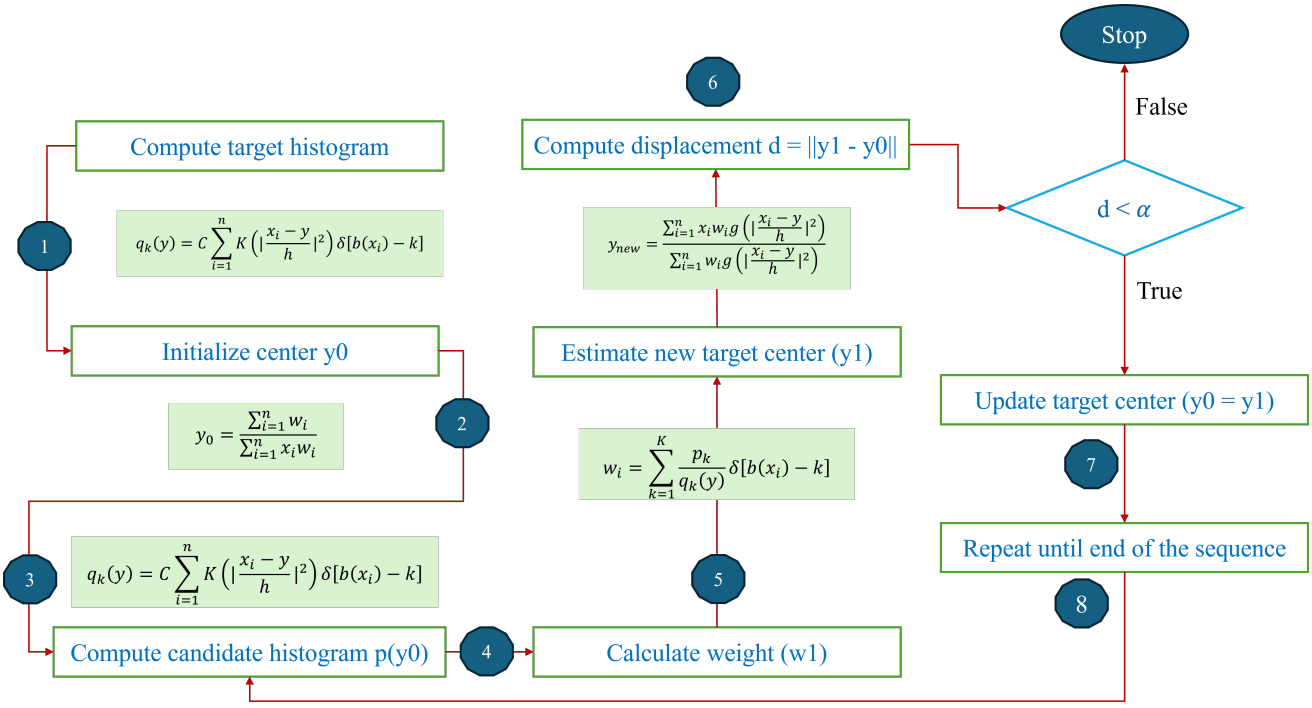
trong đó $g(\cdot)$ là đạo hàm của kernel function. Vector dịch chuyển này cho biết hướng mà cửa sổ tìm kiếm cần di chuyển để tăng mức độ tương đồng giữa hai histogram.

3.7 Iterative Optimization

Thuật toán mean shift thực hiện cập nhật vị trí theo quy tắc trên cho đến khi hội tụ:

$$y \leftarrow y_{new} \tag{29}$$

Quá trình lặp dừng khi sự thay đổi của vị trí nhỏ hơn một ngưỡng định trước. Quy trình tổng quát của mean shift tracking có thể được mô tả như sau:



Hình 7: Tổng quát các bước tính toán của Mean Shift Tracking

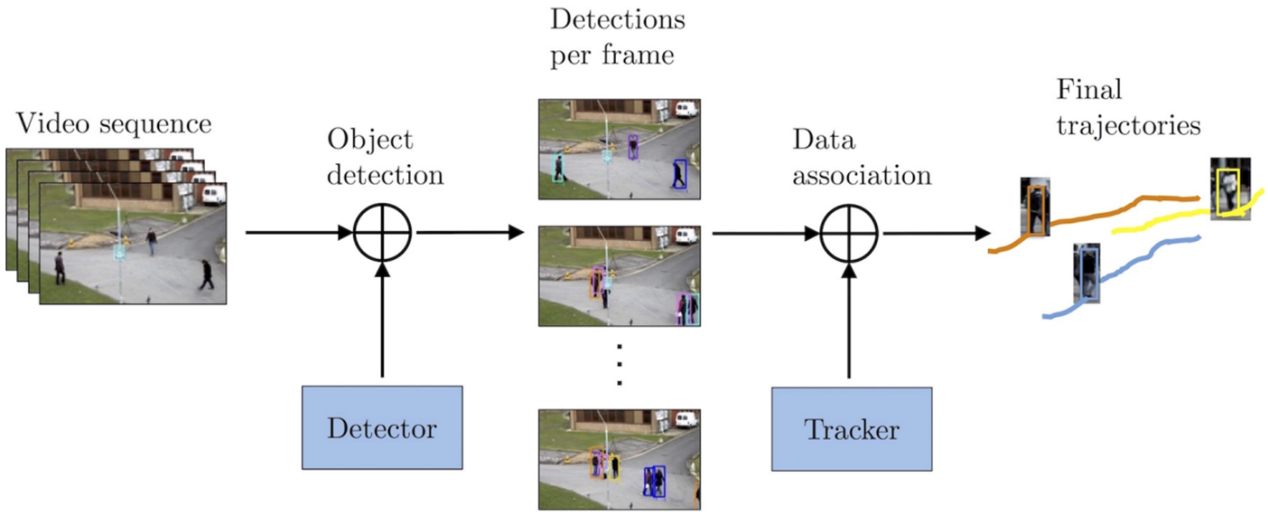
3.8 Thảo luận

Mean shift tracking có ưu điểm là không cần quét toàn bộ vùng tìm kiếm mà chỉ thực hiện tối ưu hóa lặp để tìm vị trí tốt nhất. Điều này giúp thuật toán đạt hiệu quả tính toán tốt hơn so với nhiều phương pháp classical tracking.
Tuy nhiên, mean shift chỉ tìm cực đại cục bộ của hàm tương đồng. Nếu khởi tạo ban đầu không đủ gần vị trí thực của đối tượng, thuật toán có thể hội tụ về một cực đại sai. Ngoài ra, phương pháp này cũng gặp khó khăn khi đối tượng thay đổi mạnh về ngoại hình hoặc bị che khuất.

III Multiple Object Tracking (Tracking by Detection)

1 Tổng quan

Trong nhiều ứng dụng thực tế như giám sát giao thông, phân tích hành vi đám đông, hoặc hệ thống giám sát an ninh, hệ thống cần theo dõi đồng thời nhiều đối tượng trong video. Bài toán này được gọi là *Multiple Object Tracking (MOT)*. Không giống như *Single Object Tracking*, nơi chỉ có một đối tượng cần theo dõi, MOT phải xử lý nhiều thách thức phức tạp hơn, bao gồm sự xuất hiện hoặc biến mất của đối tượng, hiện tượng che khuất (*occlusion*), cũng như sự tương tác và giao cắt giữa các đối tượng.
Một trong những cách tiếp cận phổ biến nhất hiện nay cho bài toán MOT là *Tracking-by-Detection*. Ý tưởng chính của phương pháp này là tách bài toán thành hai bước: đầu tiên phát hiện đối tượng trong từng khung hình bằng một mô hình *Object Detection*, sau đó liên kết (*associate*) các phát hiện này qua các khung hình liên tiếp để xây dựng quỹ đạo chuyển động của từng đối tượng.
Cụ thể, với mỗi khung hình của video, một bộ phát hiện đối tượng sẽ tạo ra một tập các bounding box tương ứng với các đối tượng được phát hiện. Tuy nhiên, các bounding box này chỉ biểu diễn trạng thái của đối tượng trong từng khung hình riêng lẻ. Để theo dõi đối tượng theo thời gian, hệ thống cần xác định bounding box nào ở khung hình hiện tại tương ứng với đối tượng nào ở khung hình trước đó. Quá trình này được gọi là *data association*. Một hệ thống Multiple Object Tracking theo mô hình



Hình 8: Tracking Pipeline với Multiple Objects Tracking

Tracking-by-Detection thường bao gồm hai thành phần quan trọng. Thành phần thứ nhất là *motion model*, được sử dụng để dự đoán vị trí của đối tượng trong khung hình tiếp theo dựa trên trạng thái trước đó. Thành phần thứ hai là *data association*, có nhiệm vụ ghép nối các dự đoán của tracker với các phát hiện mới từ detector. Hai thành phần này phối hợp với nhau để duy trì danh sách các quỹ đạo (*tracks*) của đối tượng trong toàn bộ chuỗi video.
Tổng thể pipeline của một hệ thống MOT điển hình có thể được mô tả như sau: đối với mỗi frame, hệ thống trước tiên sử dụng một object detector để tạo ra các bounding box. Sau đó, motion model dự đoán vị trí mới của các objects đã được theo dõi. Cuối cùng, thuật toán data association sẽ ghép nối các dự đoán này với các bounding box vừa được phát hiện để cập nhật trạng thái của các track hiện có hoặc khởi tạo các track mới.

2 Motion Model

Trong hệ thống Multiple Object Tracking (MOT), vị trí của các đối tượng trong video thường thay đổi liên tục theo thời gian. Tuy nhiên, các hệ thống phát hiện đối tượng (*object detectors*) không phải lúc nào cũng cung cấp kết quả ổn định: bounding box có thể bị nhiễu, bị lệch vị trí, hoặc thậm chí bị bỏ sót trong một số khung hình. Do đó, một thành phần quan trọng trong MOT là *motion model*, được sử dụng để dự đoán trạng thái của đối tượng trong khung hình tiếp theo dựa trên trạng thái đã quan sát trước đó. Việc dự đoán này giúp duy trì tính liên tục của quỹ đạo đối tượng và hỗ trợ quá trình *data association*.

2.1 State Representation

Để mô hình hóa chuyển động của đối tượng, mỗi track thường được biểu diễn bằng một vector trạng thái (*state vector*). Một biểu diễn phổ biến trong nhiều hệ thống MOT hiện đại là:

$$x = (cx, cy, a, h, \dot{cx}, \dot{cy}, \dot{a}, \dot{h})$$

trong đó:

- cx, cy: tọa độ tâm của bounding box
- a: tỉ lệ khung hình (*aspect ratio*)

- h: chiều cao của bounding box
- $\dot{cx}, \dot{cy}$: vận tốc theo trục x và y
- $\dot{a}, \dot{h}$: vận tốc thay đổi kích thước

Biểu diễn này cho phép hệ thống không chỉ mô tả vị trí hiện tại của đối tượng mà còn nắm bắt xu hướng chuyển động của nó theo thời gian.

2.2 Dynamic Motion Model

Trong nhiều hệ thống MOT, chuyển động của đối tượng thường được giả định tuân theo mô hình vận tốc không đổi (*constant velocity model*). Theo giả định này, trạng thái của đối tượng tại thời điểm t có thể được dự đoán từ trạng thái tại thời điểm $t-1$ thông qua phương trình tuyến tính:

$$x_t = Fx_{t-1} + w_t$$

trong đó:

- F: ma trận chuyển trạng thái (*state transition matrix*)
- w_t: nhiễu quá trình (*process noise*)

Ma trận F mô tả cách trạng thái của hệ thống thay đổi theo thời gian. Trong trường hợp mô hình vận tốc không đổi, F thường có dạng:

$$F = \begin{bmatrix} I & \Delta t I \\ 0 & I \end{bmatrix}$$

trong đó I là ma trận đơn vị và Δt là khoảng thời gian giữa hai khung hình liên tiếp.

2.3 Kalman Filter

Để thực hiện việc dự đoán và cập nhật trạng thái một cách hiệu quả, nhiều hệ thống MOT sử dụng *Kalman Filter*. Đây là một thuật toán ước lượng trạng thái tối ưu cho các hệ thống tuyến tính với nhiễu Gaussian. Kalman Filter duy trì hai thành phần quan trọng:

- vector trạng thái x
- ma trận hiệp phương sai P, biểu diễn độ không chắc chắn của trạng thái

Quy trình của Kalman Filter bao gồm hai bước chính: *prediction* và *update*.

Prediction Step Trong bước dự đoán, trạng thái của đối tượng được ước lượng cho khung hình tiếp theo:

$$x_t^- = Fx_{t-1}$$

Đồng thời, ma trận hiệp phương sai của trạng thái cũng được cập nhật:

$$P_t^- = FP_{t-1}F^T + Q$$

trong đó:

- x_t^-: trạng thái dự đoán trước khi quan sát
- P_t^-: độ không chắc chắn của trạng thái dự đoán
- Q: ma trận nhiễu quá trình

Bước dự đoán này cung cấp một ước lượng ban đầu về vị trí của đối tượng trong khung hình mới.

Measurement Model Khi detector cung cấp một detection mới, hệ thống nhận được một vector quan sát:

$$z_t = (cx, cy, a, h)$$

Quan sát này liên hệ với trạng thái thông qua phương trình:

$$z_t = Hx_t + v_t$$

trong đó:

- H: ma trận quan sát
- v_t: nhiễu đo lường (*measurement noise*)

Update Step Sau khi có quan sát mới, Kalman Filter cập nhật trạng thái thông qua Kalman gain:

$$K_t = P_t^- H^T (H P_t^- H^T + R)^{-1}$$

trong đó R là ma trận hiệp phương sai của nhiễu đo lường. Trạng thái sau cập nhật được tính như sau:

$$x_t = x_t^- + K_t(z_t - Hx_t^-)$$

Đồng thời, ma trận hiệp phương sai được cập nhật:

$$P_t = (I - K_t H)P_t^-$$

Bước cập nhật này kết hợp thông tin từ dự đoán chuyển động và quan sát thực tế, giúp giảm sai số trong ước lượng trạng thái.

3 Data Association

Sau khi dự đoán vị trí của các đối tượng thông qua motion model, hệ thống cần xác định bounding box nào trong các detection mới tương ứng với mỗi track đang tồn tại. Quá trình này được gọi là *data association*, và đây là một trong những thành phần quan trọng nhất của Multiple Object Tracking.
Giả sử tại thời điểm t, hệ thống có một tập các track đã tồn tại:

$$T = \{T_1, T_2, ..., T_n\}$$

và một tập các detection mới từ object detector:

$$D = \{D_1, D_2, ..., D_m\}$$

Mục tiêu của data association là tìm ánh xạ giữa các track và các detection sao cho mỗi detection được gán cho track phù hợp nhất. Để thực hiện điều này, hệ thống thường xây dựng một *cost matrix* C, trong đó mỗi phần tử C_{ij} biểu diễn chi phí khi ghép track T_i với detection D_j. Một cách phổ biến để tính chi phí là sử dụng độ chồng lấp giữa hai bounding box, được đo bằng chỉ số *Intersection over Union (IoU)*:

$$C_{ij} = 1 - IoU(B_i, B_j)$$

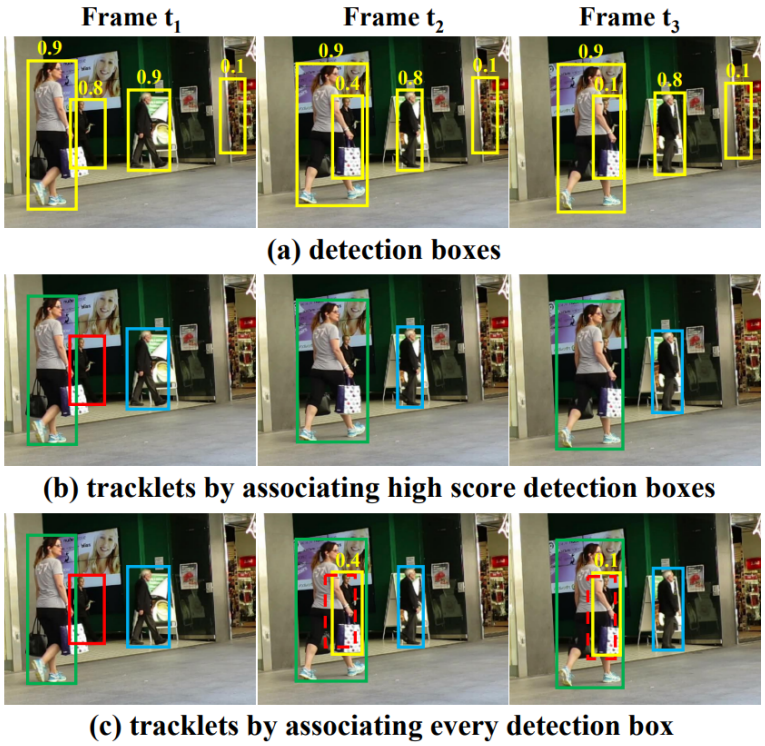
trong đó B_i là bounding box dự đoán của track T_i, và B_j là bounding box của detection D_j.
Sau khi xây dựng cost matrix, bài toán ghép nối có thể được biểu diễn như một bài toán tối ưu tổ hợp. Một thuật toán phổ biến để giải bài toán này là *Hungarian Algorithm*, cho phép tìm phép gán tối ưu giữa các track và detection sao cho tổng chi phí là nhỏ nhất.
Sau bước association, các track và detection được chia thành ba nhóm: các cặp track-detection được ghép thành công, các track không tìm được detection tương ứng, và các detection chưa được gán cho track nào. Các track được ghép thành công sẽ được cập nhật trạng thái bằng detection mới. Các track không được ghép có thể được giữ lại trong một số khung hình để xử lý trường hợp detector bỏ sót. Ngược lại, các detection chưa được gán thường được sử dụng để khởi tạo các track mới.
Thông qua sự kết hợp giữa motion model và data association, hệ thống Multiple Object Tracking có thể duy trì và cập nhật quỹ đạo của nhiều đối tượng trong suốt chuỗi video, ngay cả trong những tình huống phức tạp như giao cắt hoặc che khuất giữa các đối tượng.

IV Modern MOT Algorithms

1 ByteTrack

ByteTrack là một phương pháp theo dõi đa đối tượng (Multi-Object Tracking - MOT) tiên tiến, giúp cải thiện độ chính xác và tính ổn định của việc theo dõi đối tượng trong video. Phương pháp này tập trung vào việc sử dụng toàn bộ các hộp phát hiện (bounding box), thay vì chỉ giữ lại các hộp có điểm số cao như các phương pháp truyền thống. Điều này giúp giảm thiểu tình trạng mất dấu đối tượng do bị che khuất hoặc mờ chuyển động.
Phương pháp chính: ByteTrack sử dụng hai giai đoạn ghép nối dữ liệu để liên kết các đối tượng qua từng khung hình:

- **Ghép nối hộp phát hiện có điểm số cao:** Các hộp có độ tin cậy cao được liên kết với tracklets hiện có bằng thuật toán Hungarian, sử dụng độ tương đồng IoU hoặc Re-ID.
- **Ghép nối hộp phát hiện có điểm số thấp:** Các tracklets chưa được ghép nối tiếp tục kiểm tra với các hộp phát hiện có điểm số thấp để khôi phục những đối tượng bị mất dấu. Chỉ những hộp không liên kết được mới bị loại bỏ.



Hình 9: So sánh phương pháp truyền thống và ByteTrack trong việc liên kết các bounding box.
Link

Hình 9 cho thấy sự khác biệt giữa phương pháp truyền thống và ByteTrack trong việc xử lý các hộp phát hiện (detection boxes). (a) Hiển thị tất cả các hộp phát hiện với điểm số dự đoán. Trong phương pháp truyền thống, chỉ những hộp có điểm số cao mới được liên kết với đường đi của đối tượng, dẫn đến tình trạng mất dấu khi đối tượng bị che khuất hoặc khi hộp phát hiện có điểm số thấp (Hình 3(b)). Ngược lại, ByteTrack tận dụng toàn bộ các hộp phát hiện, bao gồm cả những hộp có điểm số thấp, giúp duy trì theo dõi liên tục ngay cả khi đối tượng tạm thời bị che khuất. Để nâng cao độ chính xác trong việc liên kết các đường đi, ByteTrack sử dụng Bộ lọc Kalman nhằm dự đoán vị trí tiếp theo của đối tượng, đảm bảo khả năng theo dõi ổn định ngay cả khi có sự thay đổi về tốc độ hoặc hướng di chuyển(Hình 9(c)).

2 DeepSORT

DeepSORT (Deep Simple Online and Realtime Tracker) là phiên bản cải tiến của thuật toán SORT, giúp theo dõi đối tượng đa mục tiêu (MOT - Multi-Object Tracking) trong thời gian thực với độ chính xác cao hơn. Khác với SORT, DeepSORT không chỉ sử dụng thông tin chuyển động mà còn kết hợp đặc trưng ngoại hình (appearance) để duy trì nhận diện đối tượng ngay cả khi bị che khuất hoặc khi có nhiều đối tượng giống nhau.
Phương pháp chính: DeepSORT sử dụng bộ lọc Kalman (Kalman filtering) để dự đoán vị trí tiếp theo của đối tượng, sau đó ghép nối các hộp phát hiện với đường đi hiện tại bằng thuật toán Hungarian, dựa trên hai tiêu chí:

- **Thông tin chuyển động:** Được đo bằng khoảng cách Mahalanobis (Mahalanobis distance), đánh giá mức độ phù hợp giữa dự đoán của Kalman và quan sát mới.

$$d^{(1)}(i,j) = (d_j - y_i)^T S_i^{-1} (d_j - y_i) \tag{30}$$

  Trong đó:

  - d_j là vị trí hộp giới hạn của quan sát thứ j.
  - y_i là dự đoán từ bộ lọc Kalman cho tracklet thứ i.
  - S_i là ma trận hiệp phương sai của dự đoán.

- **Thông tin ngoại hình:** Được đo bằng khoảng cách Cosine, giúp so sánh đặc trưng ngoại hình trích xuất từ mạng nơ-ron tích chập (CNN) để nhận diện đối tượng ngay cả khi bị che khuất.

$$d^{(2)}(i,j) = \min\left\{1 - r_j^T r_k(i) \mid r_k(i) \in R_i\right\} \tag{31}$$

  Trong đó:

  - r_j là đặc trưng ngoại hình (appearance feature vector) của hộp phát hiện thứ j.
  - R_i là tập hợp đặc trưng ngoại hình của tracklet thứ i.

V Giới thiệu về bài tập Object Tracking & Counting

Trong bài tập này, chúng ta xây dựng một hệ thống đếm và theo dõi phương tiện giao thông dựa trên video, ứng dụng các công nghệ hiện đại trong Computer Vision để giải quyết bài toán nhận diện, theo dõi và đếm xe một cách tự động. Cụ thể, hệ thống sẽ kết hợp ba thành phần chính: YOLO26, ByteTrack, và DeepSORT.

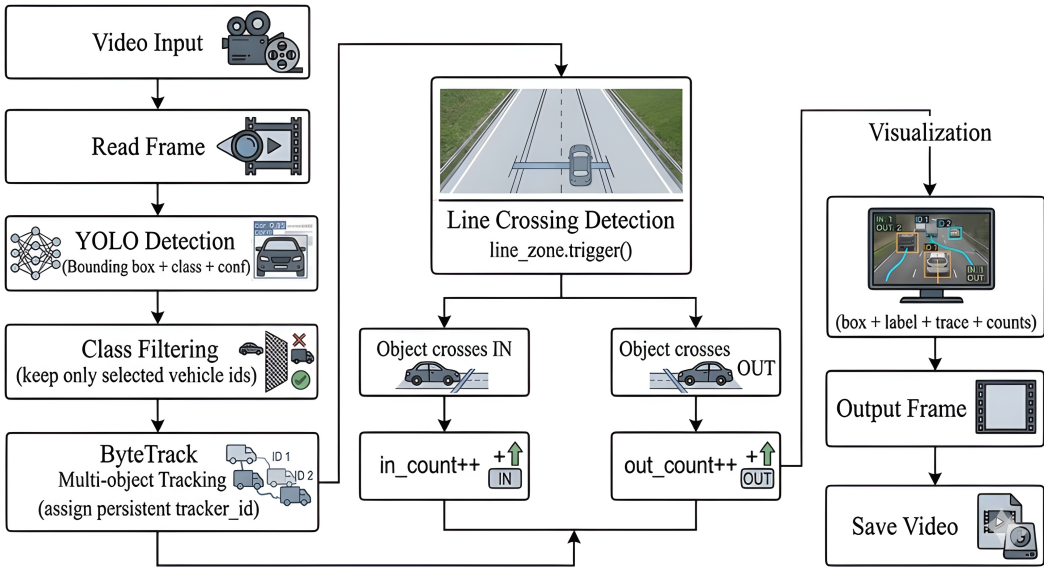
- **YOLO26**: một mô hình phát hiện đối tượng hiện đại được phát triển nhằm tối ưu hiệu suất phát hiện trong các hệ thống thị giác máy tính thời gian thực. YOLO26 kế thừa triết lý thiết kế của các phiên bản YOLO trước đó nhưng được cải tiến về kiến trúc backbone, cơ chế trích xuất đặc trưng và chiến lược huấn luyện, giúp nâng cao độ chính xác cũng như khả năng tổng quát hóa trên nhiều tập dữ liệu khác nhau. Bên cạnh đó, YOLO26 được thiết kế với mục tiêu tối ưu tốc độ suy luận, cho phép triển khai hiệu quả trong các hệ thống xử lý video như giám sát giao thông hoặc phân tích hành vi. Trong bài toán Object Tracking và Counting, YOLO26 đóng vai trò là bộ phát hiện đối tượng (*detector*), cung cấp các bounding box của phương tiện trong từng frame, làm đầu vào cho các thuật toán theo dõi đối tượng ở các bước tiếp theo.

- **ByteTrack**: Thuật toán được áp dụng để duy trì tính liên tục trong việc theo dõi các đối tượng qua các frame. Như đã phân tích ở trên, ByteTrack cải thiện độ chính xác bằng cách tận dụng toàn bộ các hộp phát hiện (bounding box), bao gồm cả những hộp có điểm số thấp, để giảm thiểu tình trạng mất dấu đối tượng khi bị che khuất hoặc chuyển động mờ. ByteTrack sử dụng thuật toán Hungarian và Bộ lọc Kalman để liên kết các đối tượng qua các frame, đảm bảo tính ổn định ngay cả trong môi trường giao thông phức tạp.

- **Deep SORT**: Để nâng cao khả năng theo dõi, đặc biệt trong các tình huống có nhiều đối tượng giống nhau hoặc bị che khuất, DeepSORT được tích hợp vào hệ thống. DeepSORT không chỉ dựa vào thông tin chuyển động mà còn sử dụng đặc trưng ngoại hình (appearance features) để duy trì danh tính của từng đối tượng. Điều này giúp hệ thống duy trì theo dõi ngay cả khi có sự giao cắt hoặc tạm thời mất dấu. Phương pháp này đã được giải thích ở phần Kiến thức liên quan, với việc sử dụng khoảng cách Mahalanobis để đánh giá mức độ phù hợp giữa dự đoán và quan sát, cũng như khoảng cách Cosine để so sánh đặc trưng ngoại hình.

Quá trình thực hiện bao gồm các bước sau:

1. **Phát hiện đối tượng**: Áp dụng YOLO26 để nhận diện và gán nhãn cho các xe trong mỗi frame của video. Các đối tượng được lọc theo các lớp cần quan tâm: `car`, `motorcycle`, `bus` và `truck`.



Hình 10: Sơ đồ biểu diễn luồng logic cho việc đếm và hiển thị kết quả xe.

2. **Theo dõi bằng ByteTrack**: Sử dụng ByteTrack để liên kết các đối tượng qua các frame dựa trên thông tin vị trí và độ tin cậy, từ đó cập nhật và duy trì vị trí của từng xe theo thời gian.

3. **Theo dõi bằng Deep SORT**: Áp dụng Deep SORT để cải thiện khả năng theo dõi, đảm bảo việc duy trì danh tính của từng xe ngay cả khi có hiện tượng giao nhau hay tạm mất dấu.

4. **Đếm xe và hiển thị kết quả**: Kết quả được chú thích trực tiếp lên video thông qua các bounding box, nhãn chứa thông tin tên xe và số hiệu (ID) của từng đối tượng. Hệ thống cũng đánh dấu một khu vực xác định (line zone) để tiến hành đếm số lượng xe đi qua.

1 Giới thiệu các thư viện

1.1 Ultralytics

Thư viện **ultralytics** là một thư viện Python chuyên về các mô hình deep learning, đặc biệt nổi bật với dòng mô hình YOLO (You Only Look Once) cho nhiệm vụ nhận diện đối tượng.

- **Tính năng nổi bật:** Cung cấp các mô hình tiên tiến như YOLO26, hỗ trợ huấn luyện, đánh giá và triển khai các mô hình phát hiện đối tượng với độ chính xác cao và tốc độ xử lý nhanh.
- **Ứng dụng:** Phù hợp cho các bài toán về nhận diện đối tượng trong hình ảnh và video, giám sát an ninh, phân tích giao thông, và nhiều lĩnh vực khác.

1.2 Supervision

Thư viện **supervision** là một công cụ mạnh mẽ hỗ trợ xử lý và chú thích video, giúp người dùng dễ dàng tích hợp kết quả từ các mô hình nhận diện (như YOLO) vào quy trình xử lý hậu kỳ.

- **Tính năng nổi bật:** Cung cấp các hàm tiện ích để vẽ bounding box, nhãn, trace (đường đi) và xử lý video theo dạng frame-by-frame.
- **Ứng dụng:** Giúp hiển thị kết quả phát hiện và theo dõi đối tượng một cách trực quan, hỗ trợ phát triển các hệ thống đếm xe, giám sát giao thông, và phân tích video.

1.3 Deep SORT realtime

Thư viện **Deep SORT realtime** triển khai thuật toán Deep SORT (Simple Online and Realtime Tracking) cho việc theo dõi đa đối tượng trong thời gian thực.

- **Tính năng nổi bật:** Kết hợp các đặc trưng trực quan và thuật toán sắp xếp để duy trì danh tính của các đối tượng qua các frame của video. Điều này giúp hệ thống theo dõi được nhiều đối tượng cùng lúc với độ chính xác cao.
- **Ứng dụng:** Được sử dụng trong các hệ thống theo dõi đối tượng như giám sát an ninh, phân tích giao thông, và các ứng dụng cần duy trì danh tính của đối tượng qua thời gian.

2 Ví dụ triển khai

Code triển khai cho ví dụ trên: **Tại đây**.



Hình 11: Kết quả của thuật toán DeepSORT

VI Phụ Lục

1. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc hoặc quá trình triển khai project trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được hỗ trợ và giải đáp trong thời gian sớm nhất.



Hình 12: Hình ảnh group facebook AIO Q&A.

2. **Rubric:**

Object Tracking and Counting		
Câu	**Kiến thức**	**Tiêu chí đánh giá**
1	- Hiểu bài toán **Object Detection** trong video. - Nắm được nguyên lý hoạt động của các mô hình phát hiện đối tượng hiện đại như YOLO. - Hiểu cách sử dụng detector để phát hiện phương tiện giao thông trong từng frame.	- Triển khai thành công mô hình phát hiện đối tượng (YOLOv8 hoặc YOLO26). - Xác định chính xác bounding box của các phương tiện như `car`, `motorcycle`, `bus`, `truck`.
2	- Hiểu nguyên lý **Motion Model** trong Multiple Object Tracking. - Nắm được cách biểu diễn trạng thái đối tượng và vai trò của Kalman Filter trong dự đoán chuyển động.	- Triển khai cơ chế dự đoán vị trí đối tượng giữa các frame. - Sử dụng Kalman Filter để cập nhật trạng thái của các track theo thời gian.
3	- Hiểu bài toán **Data Association** trong Multiple Object Tracking. - Hiểu được cost matrix và cơ chế ghép nối giữa track và detection.	- Áp dụng thuật toán ghép nối (ví dụ: Hungarian Algorithm). - Duy trì ID ổn định cho các đối tượng khi di chuyển qua nhiều frame.
4	- Hiểu pipeline hoàn chỉnh của hệ thống **Multiple Object Tracking và Object Counting**. - Nắm được cơ chế đếm đối tượng dựa trên line crossing hoặc vùng quan tâm.	- Xây dựng hệ thống theo dõi phương tiện trong video. - Thực hiện đếm số lượng phương tiện đi qua đường đếm và hiển thị kết quả theo thời gian thực.